



Large-Scale Analysis of Cryptographic Attacks on Internet Public Key Infrastructure

handed in by

Petro Novotnyy

E-mail: 2nov.petro@gmail.com

Matriculation number: 459812

from Fakultät IV - Elektrotechnik und Informatik
der Technischen Universität Berlin
zur Erlangung des akademischen Grades

MASTER OF SCIENCE (M.Sc.)

Supervisors:

Reviewer: Prof. Dr. Jean-Pierre Seifert, Technische Universität Berlin
Reviewer: Prof. Dr. Marian Margraf, Freie Universität Berlin
Supervisor: Dr. Henning Seidler and Sebastian Neef, Technische Universität Berlin

January 2, 2026

Berlin

Declaration

Hiermit versichere ich, dass ich die vorliegende Arbeit ohne Hilfe Dritter und ausschließlich unter Verwendung der aufgeführten Quellen und Hilfsmittel angefertigt habe. Alle Stellen die den benutzten Quellen und Hilfsmitteln wörtlich oder inhaltlich entnommen sind, habe ich als solche kenntlich gemacht.

Sofern KI-Tools verwendet wurden, habe ich Produktnamen, Hersteller, die jeweils verwendete Softwareversion und die Art der Nutzung (z.B. sprachliche Überprüfung und Verbesserung der Texte, systematische Recherche) benannt. Ich verantworte die Auswahl, die Übernahme und sämtliche Ergebnisse des von mir verwendeten KI-generierten Outputs vollumfänglich selbst.

Die Satzung zur Sicherung guter wissenschaftlicher Praxis an der TU Berlin vom 15. Februar 2023. https://www.static.tu.berlin/fileadmin/www/10002457/K3-AMBI/Amtsblatt_2023/Amtliches_Mitteilungsblatt_Nr._16_vom_30.05.2023.pdf habe ich zur Kenntnis genommen.

Ich erkläre weiterhin, dass ich die Arbeit in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegt habe.

Berlin, January 2, 2026

Petro Novotnyy

E-mail:

2nov.petro@gmail.com

Matriculation number:

459812

Abstract

RSA based Web Public Key Infrastructure allows implementation flaws to compromise its security despite correct underlying mathematics. We present an analysis of RSA vulnerabilities, examining over 11.6 billion Internet certificates totaling 8.9 billion unique public keys. Our results show that classical attacks such as Fermat factorization and ROCA detection yield diminishing returns, while greatest common divisor techniques remain as a vital method for compromising keys. GCD however comes at an unrealizable cost for exhaustive scanning, which limits it to a targeted threat vector. Overall, our results show that modern RSA failures stem primarily from rare but correlated key-generation weaknesses.

Zusammenfassung

Die RSA-basierte Web-Public-Key-Infrastruktur erlaubt es, dass Implementierungsfehler ihre Sicherheit trotz korrekter zugrunde liegender Mathematik gefährden. In dieser Arbeit präsentieren wir eine Analyse von RSA-Schwachstellen und untersuchen über 11,6 Milliarden Internet-Zertifikate mit insgesamt 8,9 Milliarden eindeutiger Schlüssel. Unsere Ergebnisse zeigen, dass klassische Angriffe wie die Fermat-Faktorisierung und die ROCA-Erkennung nur noch abnehmende Erträge liefern, während Greatest-Common-Divisor-Techniken weiterhin eine zentrale Methode zur Gefährdung von Schlüsseln darstellen. GCD-basierte Verfahren sind jedoch mit einem für vollständige Scans nicht realisierbaren Rechenaufwand verbunden, was ihren Einsatz auf gezielte Angriffsvektoren beschränkt. Insgesamt zeigen unsere Ergebnisse, dass moderne RSA-Fehler primär auf seltene, aber korrelierende Schwächen bei der Schlüsselgenerierung zurückzuführen sind.

Contents

Declaration	iii
1 Introduction	1
2 Preliminaries	3
2.1 Certificate-Based Authentication on the Web	3
2.2 TLS/SSL Protocol	5
2.2.1 Protocol Evolution	5
2.3 Implications of a Compromised Private Key	6
2.4 Cryptographic Keys	7
2.4.1 RSA	7
2.4.2 DSA	7
2.4.3 ECDSA	7
2.4.4 Other Algorithms	8
2.5 Cryptographic Attacks	8
2.5.1 RSA	8
2.5.2 DSA	9
2.5.3 ECDSA	10
2.6 Dataset	10
2.6.1 Shodan	10
2.6.2 Censys	11
3 System	13
3.1 Overview	13
3.2 Downloading the Dataset	14
3.3 Deduplicating RSA	15
3.4 Attack Pipeline	16
3.5 GCD Pipeline	16
3.6 Optimization	17
4 Experiment	19
4.1 Downloading Certificates	19

4.2	Deduplicating RSA	20
4.3	GCD	21
4.4	Attacks	22
4.5	Statistics	24
4.6	Proof of Concept faked Certificate	25
5	Discussion	27
5.1	Relevance of Singular Attacks	27
5.2	Effectiveness and Limitations of GCD-Based Analysis	28
5.3	Targeted GCD as a Practical Threat Model	28
5.4	Optimization	29
5.5	Web PKI Implications	29
5.6	Datasets	30
5.7	ECDSA	30
5.8	Ethics	30
5.9	Outlook	31
6	Conclusion	33
	Bibliography	35
A	Appendix	41
A.1	System architecture	41
A.1.1	Downloader	41
A.1.2	Deduplicator	41
A.1.3	Singular Attack Pipeline	42
A.1.4	GGT Pipelines	44

Introduction

Public-key cryptography using RSA in particular, remains a cornerstone of the Public Key Infrastructure (PKI) which is fundamental to web security. The trust in RSA relies not only on sound mathematical assumptions but also on correct key generation. Previously discovered issues such as the Debian OpenSSL vulnerability [Bel08], widespread shared prime factors in RSA moduli [Len+12], and the ROCA vulnerability [Nem+17] have demonstrated that key generation flaws can undermine RSA at an ecosystem scale. Since these disclosures, substantial progress has been made in key generation practices, certificate validation, and automated issuance, while certificate lifetimes have been reduced and monitoring infrastructures have improved [Dav25].

At the same time, the rapid growth of the Web PKI poses new challenges for vulnerability assessment: factorization-based attacks are computationally expensive, exhaustive GCD-based analyses scale poorly, and it remains unclear which known attack classes still pose a threat to modern, in-use certificates. This work addresses these challenges by performing a large-scale analysis of RSA certificates drawn from the Censys dataset, comprising over 11.6 billion certificates. We begin by deduplicating public keys followed by evaluating multiple low cost attacks including Fermat factorization, ROCA detection, small-prime tests, and blocklist lookups. Finally, we apply GCD-based techniques to validate their effectiveness at scale and evaluate the results.

Beyond measuring the amount of affected certificates, we analyze the computational feasibility of GCD attacks, simulate the impact of compromised certificates through a laboratory-based man-in-the-middle demonstration, and discuss the implications of shrinking certificate lifetimes and evolving key-generation practices.

We are not aware of any work which applies attacks at a similar order of magnitude to the 11.6 billion RSA certificates we analyzed. This includes prior work, notably *Mining Your Ps and Qs* by Heninger et al., which analyzed approximately 11.7 million RSA keys [Hen+12], and the foundational study by Lenstra et al. examining a few million public keys collected from the Internet [Len+12]. Building on these earlier studies, we show that singular per-certificate attacks that exploit known issues such as the OpenSSL Bug [Bel08] yield diminishing returns, while GCD-based

techniques remain the dominant source of newly discovered RSA compromises at scale.

However, GCD is constrained by prohibitive computational costs that limit its applicability to targeted threat models rather than exhaustive scanning. We analyze these results by examining how changes such as automated issuance, reduced certificate lifetimes, and increasing key sizes affect both attacker feasibility and defensive measures. Finally, we address the ethical implications of large-scale cryptographic vulnerability analysis, including the use of strictly passive measurement methodologies, the avoidance of active exploitation, the handling of disclosure boundaries, and the trade-offs between security analysis and potential consequences for certificate holders.

Our contributions can be found under the following link:

<https://github.com/orgs/Censys-Analysis/repositories>

Preliminaries

In this section, we will summarize relevant concepts for internet safety and detail security weak points that are relevant for the work.

2.1 Certificate-Based Authentication on the Web

Web authentication and transport security are largely built upon the Public Key Infrastructure (PKI). The PKI is a framework that allows for safe creation, management, and usage of digital certificates. Digital certificates bind an entity to a cryptographic key. This is achieved by having an issuer, referred to as a Certificate Authority (CA), sign a structured set of key-value pairs containing information about the holder (aforementioned entity and its cryptographic key), called a certificate. When the holder entity (a website) is interacted with by a user, the certificate is presented. The certificate conveys credibility by relying on the trust in the issuer (CA).

Web certificates are a specific use of digital certificates, commonly following the X.509 standard. The X.509 standard defines the previously mentioned key-value store. It is a structured, ASN.1-encoded data format containing identity and key material. Included is the holder entity's public key and identifying information such as the domain name or organization. After a CA uses its private key to sign this information, the certificate is considered protected against undetected modification, since faking a signature is deemed computationally infeasible. [[@Boe+08](#)] [[@PPR01](#)]

Since the CA uses a cryptographic key for signing, it needs a certificate to bind this key with the CA's identity. The CA's certificate is therefore issued by a different CA. This creates a chain of certificates. When a user visits a website secured with a web certificate, their browser can verify the presented certificate by walking up the chain until a trusted CA is reached. At the beginning of such chains are root CAs, these are inherently trusted by browsers and operating systems. This system (PKI) relies on the security and credibility of the globally acknowledged root CAs. If the root CA is trusted, then so are the certificate chains stemming from it. [[@PPR01](#)] [[@Hes+05](#)]

The X.509 Standard defines the holder's public key and details. Additionally, the following are defined: the *Version* defines the format version, with version 3 (X.509 v3) being most common. The *Serial Number* is a unique identification number assigned by the CA for tracking purposes. The *Issuer* attribute identifies the CA that issued this certificate. The *Validity* fields define the timeframe in which this certificate is valid. [[@Boe+08](#)]

X.509 v3 allows the CA to optionally define certificate usage. The *Subject Alternative Name (SAN)* lists additional domain names or IP addresses associated with the certificate, allowing a single certificate to possess multiple identifiers. The *Key Usage* and *Extended Key Usage (EKU)* define the permitted uses of the public key, such as server/client authentication or digital signatures. The *Basic Constraints* attribute indicates whether the certificate belongs to a CA and, if so, whether it can issue further certificates. Finally, the *CRL Distribution Points (CRL DP)* and *Authority Information Access (AIA)* provide information on where clients can check the certificates revocation status. The certificate concludes with the CA's *Signature* tamper proofing the information. [[@Boe+08](#)]

Certificate Authorities (CAs) verify the applicant's identity before awarding a certificate. Strictness depends on certificate type and usage. *Domain Validation* requires validation of domain control. *Organization Validation* verifies organization details such as name and legal registration. *Extended Validation* adds additional legal and operational checks. The CA can revoke a certificate via Certificate Revocation Lists or using the Online Certificate Status Protocol. This is commonly needed when a certificate is compromised. [[@PPR01](#)]

Certificate Authorities have a hierarchical structure. Operating systems and browsers come with self-signed Root CA certificates pre-installed. Root CAs don't interact with end entities, but issue certificates to intermediate CAs. Intermediate CAs then issue certificates to either other intermediates or to end holders. Using intermediaries limits root key exposure, thereby limiting the effect of compromised CA keys to a smaller subset of entities. CAs are subject to strict monitoring and regulation due to the significant break of trust in case their certificate gets compromised. [[@Hes+05](#)] [[@San+13](#)]

2.2 TLS/SSL Protocol

The standard protocol for establishing secure communication is called Transport Layer Security (TLS), formerly known as Secure Sockets Layer (SSL). Although SSL is now obsolete, the term "SSL" is often used interchangeably with TLS.

TLS is used to ensure the three main security guarantees. *Confidentiality* is guaranteed by encrypting all transmitted information. *Integrity* comes from the usage of message authentication codes or authenticated encryption, which prevent message modification. *Authentication* comes from the PKI and leveraged trust in the root CAs.

The parameters for secure communication are set by the TLS handshake [Res18a]:

ClientHello: The client initiates the connection by sending its supported TLS versions and cipher suites, as well as a randomly generated nonce.

ServerHello: The server selects a compatible TLS version and cipher suite from the ones provided by the client and replies with them, along with its web certificate and a different randomly generated nonce.

Verification: The client verifies the web certificate by checking the validity, revocation status and the CA chain.

Key Exchange: Depending on the cipher suite, different methods are used.

RSA exchange (deprecated): The client encrypts a pre-master secret with the server's RSA public key from the web certificate.

Diffie-Hellman (DH) / Elliptic Curve Diffie-Hellman (ECDH): The server generates a temporary DH keypair for each handshake, which provides forward secrecy. Forward secrecy protects previous sessions in case the server's private key gets compromised.

Encrypted Communication: All subsequent communication is encrypted using the negotiated session key.

2.2.1 Protocol Evolution

TLS/SSL has evolved over time further improving security and fixing vulnerabilities.

SSL 2.0 (1995) and SSL 3.0 (1996): Early versions, currently insecure and deprecated [[@PT11](#)] [[@Bar+15](#)].

TLS 1.0 (1999) and TLS 1.1 (2006): Improved security over SSL, but relied on weak cipher suites; deprecated. [[@AD99](#)] [[@DR06](#)]

TLS 1.2 (2008): Supports RSA, DHE, and ECDHE key exchanges, along with newer signature algorithms; remains widely deployed but is increasingly restricted to forward-secure cipher suites. [[@RD08](#)]

TLS 1.3 (2018): Removes legacy features, enforces forward secrecy using ECDHE key exchange and allows for RSA and ECDSA signatures; current standard. [[@Res18b](#)]

Although modern TLS versions enforce forward secrecy [[@Shi07](#)], RSA public keys remain critical for authentication, meaning that weaknesses in RSA key generation still enable attacks such as man-in-the-middle interception.

2.3 Implications of a Compromised Private Key

Web security relies on the secrecy of the private key, with its compromise breaking security guarantees. Forward secrecy prevents former communication from being decrypted, but does not increase security.

Man-in-the-Middle Attacks A compromised web certificate makes a fake server indistinguishable from a real one. Using DNS poisoning, ARP spoofing, or BGP hijacking allows attackers to silently redirect traffic while maintaining the appearance of a valid, secure connection. These attacks assume an adversary capable of redirecting client traffic. [[@Shi07](#)]

Mitigation via Forward Secrecy Forward secrecy (DHE/ECDHE) limits damage by generating unique session keys for each session preventing leaked private keys from decrypting previously recorded traffic. This does not prevent the interception and decryption of future traffic. [[@Shi07](#)]

2.4 Cryptographic Keys

Cryptographic keys are a critical part of the PKI, with most security guarantees building on the infeasible computation of the private key given the public key. In this section we will detail the three main key types as well as corresponding weaknesses and attacks. We focus on RSA, as it remains dominant in Web PKI certificates (see [Table 4.1](#)).

2.4.1 RSA

RSA is one of the more well known cryptosystems. The security of RSA relies on the assumed hardness of factoring large composite integers, which is currently infeasible for key sizes used in practice (e.g., 2048 bits and above). An RSA key pair is made up of a public key (n, e) and a private key (n, d) . The modulus n is the product of two large primes p and q . e is the public exponent (usually 65537), and d satisfies $e \cdot d \equiv 1 \pmod{\phi(n)}$. Generating ciphertext c using RSA, a message m is converted using $c = m^e \bmod n$. Decryption is done using $m = c^d \bmod n$. [[@Mor+16](#)]

2.4.2 DSA

The Digital Signature Algorithm (DSA) is designed for creating and verifying digital signatures. The security of the DSA depends on the time complexity of the discrete logarithm problem (currently computationally infeasible for large bit sizes). A DSA key pair is made of a private key x and a public key $(y = g^x \bmod p)$, where p and q are large primes, and g is a generator of a subgroup of order q . Reversing x given y is the discrete logarithm problem. [[Nat23](#)]

Signing uses a nonce k and the private key x to produce a signature (r, s) . Verification requires the public key y and the domain parameters (p, q, g) . Security depends on keeping x and k secret. Reuse or leakage of k can expose the private key.

2.4.3 ECDSA

The Elliptic Curve Digital Signature Algorithm (ECDSA) adapts DSA to elliptic curve groups. This keeps the same level of security while reducing key size. An ECDSA key

pair consists of a private key d and a public key $Q = d \cdot G$, where G is a predefined base point on a curve. [Nat23]

Signing uses a nonce k and private key d to produce a signature (r, s) . Verification requires the public key Q and curve parameters. Security depends on the secrecy of d and k . ECDSA has the same issues as DSA, such as nonce reuse.

2.4.4 Other Algorithms

Besides the mentioned algorithms, other exist, but are not widely adapted. EdDSA offers Edwards elliptic curve-based signatures. Region-specific standards (like GOST), experimental post-quantum schemes such as lattice-, hash-, and code-based ones exist, but are not broadly deployed in Internet protocols.

2.5 Cryptographic Attacks

In this section we will discuss the security of the aforementioned cryptographic algorithms as well as possible attacks in case security standards are not followed.

2.5.1 RSA

The Rivest–Shamir–Adleman (RSA) algorithm is secure under proper parameter selection and implementation. There are two main attack vectors, a weak p or q used in generating n and a small private exponent d .

The Wiener Attack aims to compromise keys where a small private exponent d was chosen ($d < N^{1/4}$). Using a small d creates patterns in the public key components, which can be extracted using continued fractional approximation to the public ratio $\frac{e}{pq}$. This ratio is directly linked to an unknown k and the private exponent d . Given a sufficiently small d the list of guesses for k becomes limited enough to recover d . [Wie90]

The Coppersmith Methodology is a collection of attacks that target small unknowns r which appear modulo to N . If it is possible to define $f(r) \equiv 0 \pmod{N}$ where r is small, compared to N , then using Coppersmith would transform the problem into an integer root problem. Using lattice reduction techniques would then produce a polynomial with much smaller coefficients, leading to the recovery of r . [Cop96]

The Return of Coppersmith Attack (ROCA) exploits a predictable structure in primes generated by vulnerable RSALib implementations, enabling factorization via lattice-based Coppersmith methods. It recovers poorly generated primes using $k \times M + (65537^a \bmod M)$ where M is the product of the first n consecutive primes. This is known as CVE-2017-15361 and has affected keys generated by a vulnerable version of RSALib used in many IOT devices. [Nem+17]

The Fermat Attack attempts to guess primes which are located close to each other based on $N = (a - b) * (a + b)$ with a being the middle of p and q , and b being the offset. If the primes are numerically close, then a will be close to the square root of N . Guessing a starting at the square root would therefore recover close primes p and q , successfully compromising the key. Fermat is only effective if p and q are very close. [MVV18]

Poorly Generated Primes can be guessed. This includes small primes, for example any of the first 1000 primes or *famous* primes, such as Mersenne Primes.

Poor Randomness Generation can lead to the same prime being generated multiple times. This leads to commonly the first prime repeating across separately created keys. Discovering such keys can be done using the Greatest Common Divisor (GCD) algorithm. If two keys a and b share a common (prime) factor, then $GCD(a, b)$ would recover it. This approach is limited by having to compare each key pair individually. We can circumvent this by combining multiple keys into a large product. GCD between a single key and a product of keys would return the same results of a pairwise comparison, given the product does not already contain the single key. In case of a duplicate, GCD would return the whole key as a result which is a meaningless solution and would therefore require deduplication. If the product is too large then it can be split among files. This would require calculating the GCD within a singular file (Intra-GCD) and between files (Inter-GCD). [Ber04]

Key reuse/leak (Blocklists) can cause severe issues if the private key was leaked in a hack and uploaded online. Finding such keys depends on the amount leaked keys (grouped into a blocklist) being compared to. Equality checks can be implemented efficiently using hashed blocklists, limited primarily by I/O throughput.

2.5.2 DSA

The Digital Signature Algorithm (DSA) is secure under proper parameter selection and implementation. A key vulnerability arises if the prime p is poorly chosen. When $p - 1$ (or $(p - 1)/2$) is smooth (factoring completely into small primes) discrete

logarithms can be solved efficiently, enabling private key recovery. Proper generation uses safe primes, $p = 2q + 1$ with q prime, which ensures a subgroup of prime order and prevents small-subgroup attacks. However given the small sample size as seen in [Table 4.1](#) we will not be attempting any attacks against this key type.

Brute-force recovery of the private key x is infeasible with recommended parameter sizes ($p \geq 2048$ bits, $q \geq 224$ –256 bits). Legacy or poorly generated keys remain susceptible to exhaustive search or other structural weaknesses.

2.5.3 ECDSA

The Elliptic Curve Digital Signature Algorithm (ECDSA) is secure under correct implementation, but is vulnerable to poor nonce management. Reusing the per-message nonce k across signatures with the same private key d allows algebraic recovery of the private key. Security therefore requires generating a unique, unpredictable nonce for every signature. Due to this attack relying on additional information, besides only the public key, attempting nonce reuse attacks falls outside the scope of this work.

Brute-force recovery of d is only feasible on small or poorly chosen curves. Standardized curves such as P-256 or P-384, make an exhaustive search computationally impractical.

2.6 Dataset

For analyzing web certificates, an external dataset was used. There were two mainstream datasets for selection: Shodan and Censys.

2.6.1 Shodan

Shodan, launched in 2009 as a commercial service providing internet-wide scans with a focus on IoT devices. Data collection relies on distributed scanners which do not perform an exhaustive scan of the IPv4 space. Shodan collects a multitude of server information, including TLS certificates, partially omitting full certificate chain or key material. For the purpose of analyzing web certificates a large part of the coverage is meaningless, while some the TLS/SSL data is incomplete and partially

malformed. Shodan has an effective coverage of around 13 million certificates [[@Sho25](#)].

2.6.2 Censys

Censys, developed by the University of Michigan, is designed for research grade certificate collection. It conducts exhaustive scans of the IPv4 address space on the common ports 443 and 22. For TLS/SSL and SSH, it retrieves full certificates, key information, and protocol details. Compared to Shodan it is much more thorough and detailed, with an effective coverage of around 13 billion certificates [[@Cen25](#)]. Given this difference Censys was chosen. After deduplicating by modulus, we obtained 8.9 billion unique RSA keys, enabling scalable vulnerability analysis.

System

In this chapter we will detail the technical requirements and implementation of our processing pipelines. All pipelines were run on a dual AMD EPYC 7281 processor server (64 cores), except for Intra-GCD running on a SLURM cluster with 70 nodes, each with 32 cores.

3.1 Overview

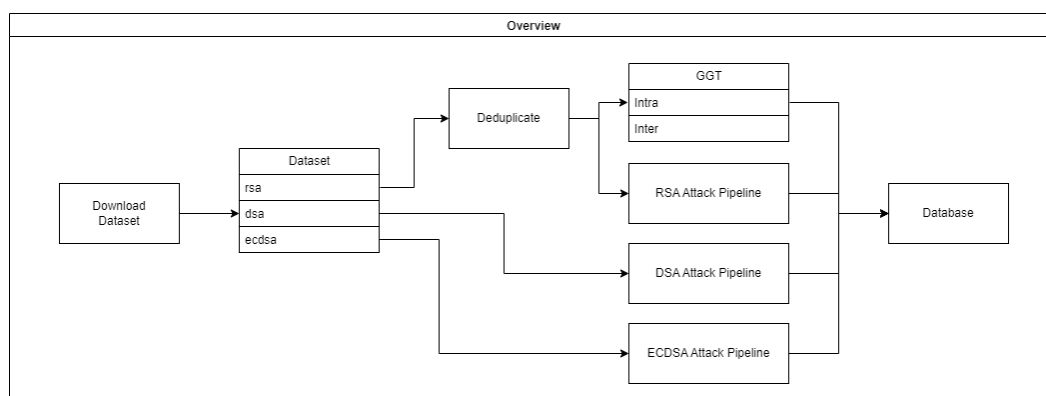


Fig. 3.1.: Pipeline Overview. Supports DSA and ECDSA analysis with a focus on RSA.

The full pipeline follows the generic purpose of downloading a dataset, analyzing it and collecting results as seen in [Figure 3.1](#). This is split into multiple parts, each responsible for a single step such as downloading, de-duplicating and testing for vulnerabilities. Although our focus is on RSA we have implemented the necessary infrastructure for DSA and ECDSA. Based on the size of the dataset (15 Billion [[@Cen25](#)]) optimization is a major focus. For this purpose we designed a generic processing pipeline visualized in [Figure 3.2](#).

It starts with a queue of file paths, which contain the input data. These paths are consumed by readers, which read and parse the file contents into required objects. These objects get fed into a multi-producer-multi-consumer (MPMC) bounded channel. Workers take objects from this channel and process them, in case of a success they push the results into a different MPMC channel from which writers save the

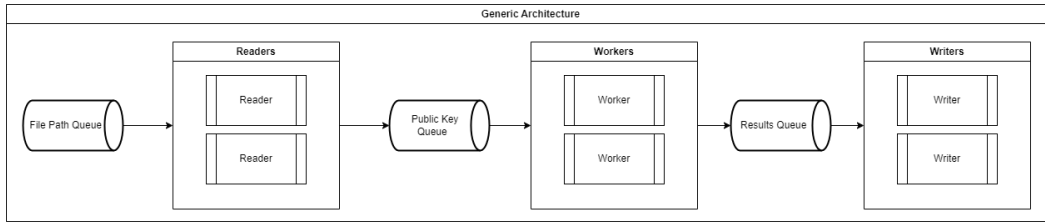


Fig. 3.2.: A Generic Pipeline. Consists of a file queue being fed into reader threads. These reader threads read files and split them into separate certificates. Each certificate gets passed through a different queue to the worker threads. Worker threads analyze or process the certificate. If the processing has a positive result, then the certificate and result both get sent to a writer to save to disk.

results to the disk. Following this structure allows us to easily increase the amount of threads working on any task in case a bottleneck appears. This is possible due to the high throughput of the queues (0.25 million RSA certificates per second). The following descriptions of single processing steps are generally based on this design.

3.2 Downloading the Dataset

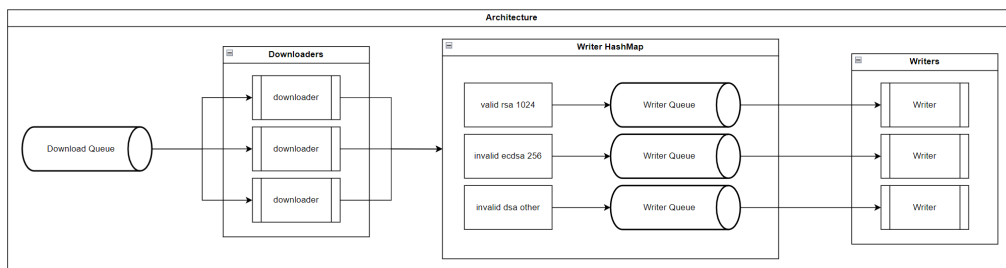


Fig. 3.3.: Downloader Pipeline. Based on the generic pipeline with a different method of sending processed certificates from workers to writers.

The Censys dataset is available for download in the .avro [Apa25] format. For our purposes, we do not require the complete certificate metadata, only the holder's public key and some identifiers. Additionally, we need to differentiate the key types used in the certificates to later distribute these to their corresponding attack pipelines. This leads us to partially sorting the dataset during download. To achieve this, we modify the connection between workers and writers by substituting one channel with multiple, as visualized in Figure 3.3. Further workflow can be seen under Appendix A.1.1. Doing so allows the workers to choose which channel to push the downloaded certificate into. Selection is based on key types, key size and validity.

Validity in our context does not mean that the certificate would be recognized as trusted, instead we base validity solely on the validity period. A self-signed certificate would be considered valid as long as the current date falls after the *not before* value and before the *not after* value defined in the certificate.

3.3 Deduplicating RSA

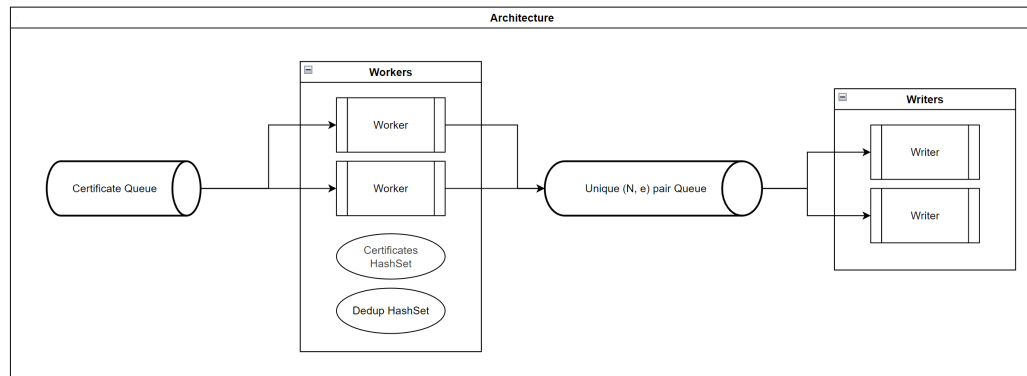


Fig. 3.4.: Deduplication pipeline. Based on the generic pipeline with an additional processing context in the workers.

In the initial testing on a smaller sample size we discovered that many RSA public keys repeat. Removing these duplicates is necessary for effective results when running GCD as well as saving computational time on the singular attacks executed in the attack pipeline. The structure follows our generic pipeline with adjustments to the workers. Given our preliminary sort, we can handle each key size and validity separately.

Loading all certificates into a hash set would be inefficient due to the largest category (invalid 2048 bit) containing 1.7TB of unique n, e pairs. We reduced this by hashing the values (SHA-1). The workers have a shared hash set where the hash of all the already seen public keys is stored. When processing a public key, its hash is first looked up in the shared set. If it is seen then it is a duplicate, if not then it is new and can be sent to the writers after its hash has been added into the shared set.

This reduced the RAM requirements down to 400GB. When handling duplicates which fall under valid and invalid, we assign them to the valid category. We first process all the **valid** keys first and then keep the **valid hash set** during the processing of the invalid hash set. When processing the invalid certificates, we do two lookups, first into the **valid hash set** and then into the invalid one. If a key was not seen

in either, then it is unique and can be saved as such. The architecture is shown in [Figure 3.4](#) and further details are shown in [Appendix A.1.2](#).

3.4 Attack Pipeline

The Attack Pipeline is based on the generic pipeline with a difference in the workers. This time when processing public keys we attempt multiple attacks in order, when any of them succeed we forward the result to the writers. Doing so saves the success of each attack even if they were all performed on the same certificate by the same worker, instead of saving just one success. Details shown in [Appendix A.1.3](#).

3.5 GCD Pipeline

As mentioned in [Section 2.5.1](#), calling GCD between each pair of certificates is not efficient. Instead, we can combine certificates into a large product and call GCD between a single certificate and the product. In our case, we build products out of files, so each product would consist of 1 million unique keys.

To achieve equivalence with a naive pairwise GCD implementation, a two-step process is required. First we need to handle the combination of certificates within the same file and then we can handle certificates from two different files. Handling certificates from different files requires, building the product of one of the files and then calling GCD with that product and the other certificate (Inter-GCD). Handling certificates from the same file requires removing a certificate from the product before GCD can be called, otherwise, as described in [Section 2.5.1](#), the result would be the full key and not its factors. This is achieved using Bernsteins Algorithm [[Ber04](#)] (Intra-GCD), which computes a product and remainder tree to quickly calculate products without a certain certificate in them.

When computing Inter-GCD we need to build products. These products are reusable, therefore we compute as many products to fill our RAM before calling GCD with each certificate and these products. Then we compute the next set of products and so on until all combinations have been executed. For Intra-GCD each file is self-sustained, therefore we just use Bernsteins Algorithm on each file separately. A detailed workflow is shown in [Appendix A.1.4](#).

3.6 Optimization

The large dataset required optimizations in processing, for this reason many benchmarks were run to determine the best implementations. We considered using a database such as Clickhouse [Cli] this however was slower than reading files from disk. To read and parse a file from disk we considered different libraries and implementations for which we concluded on `std::read` from rust, following a decompressing using `flate2` and `sonic` for parsing strings into objects to be the fastest. Crossbeam MPMC channels outperformed `tokio::mpsc` as well as native threads outperforming `tokio::spawn` and `tokio::spawn_blocking`. This could be further optimizable using techniques like `io_uring` to increase read speeds. Details about the benchmarking process can be found in the corresponding repositories at

<https://github.com/orgs/Censys-Analysis/repositories>

Experiment

The experiments were run on a dual AMD EPYC 7281 processor server (64 cores) except for Intra-GCD running on a SLURM cluster with 70 nodes, each with 32 cores. Each part of the full pipeline was separated into a docker container running rust in `-release` mode, except for the downloader running python 3.11.

4.1 Downloading Certificates

The download was started on the 30.05.2025 and is estimated to have taken around 3 weeks. It downloaded around 16 billion certificates totaling around 30TB which after processing and gzipping was reduced to 5.1TB. For our experiments only RSA certificates were required, these made up 11.5 billion certificate or 72.3% of all web certificates on Censys as see in [Table 4.1](#).

Type	Status	Count
ECDSA	Valid	1 539 025 314
	Invalid	2 917 789 530
RSA	Valid	2 453 586 325
	Invalid	9 188 275 069
DSA	Valid	28
	Invalid	3 475
Total Certificates		16 098 679 741

Tab. 4.1.: Certificate Download Summary. It shows the three types of downloaded certificates, split into valid and invalid based on the download date of 30.05.2025 and the certificates *Validity* time-frame.

Of note is the comparatively small amount of DSA certificates. Additionally, we confirm the prevalence of RSA with only 38.5% of all valid certificates being of the newer ECDSA format as seen in [Table 4.1](#). As mentioned previously, **Valid** refers to any certificate which is not expired, it does not guarantee a valid chain of trust or the certificates recognition as trusted.

4.2 Deduplicating RSA

Deduplicating RSA dropped all data except n and e . It took about 24 hours and removed around 2.5 billion certificates: 1 billion valid certificates and 1.5 billion invalid certificates, see [Table 4.2](#). This required a peak of around 400 GB RAM and an additional 2.7 TB of disk space. As part of this experiment, we discovered 9 instances where the same n was used with different e values. Upon looking into these cases, we discovered that most of them were test certificates, never having a valid chain of trust. There was one exception, which in addition to multiple test certificates, contained a certificate with a previously valid chain of trust.

Key Size	Valid			Invalid					Totals	
	Unique	Duplicates	Total	Unique	Duplicates (Self)	Duplicates (Other)	Duplicates (Total)	Total	Unique	Certificates
other	2 474	980	3 454	21 010	1 380	101	1 481	22 491	23 484	25 945
512	104 565	56 763	161 328	591 929	45 651	20 120	65 771	657 700	696 494	819 028
768	249 530	15 199 907	15 449 437	46 835	1 194 063	220 543	1 414 606	1 461 441	296 365	16 910 878
1016	303	0	303	9	0	0	0	9	312	312
1024	152 934 798	7 995 969	160 930 767	44 094 820	3 924 513	2 787 257	6 711 770	50 806 590	197 029 618	211 737 357
1032	39	41	80	159 133	34	0	34	159 167	159 172	159 247
1040	74 003	218	74 221	677 922	87 691	41 467	129 158	807 080	751 925	881 301
1048	21	0	21	264	14	0	14	278	285	299
1232	3 060	0	3 060	6	0	0	0	6	3 066	3 066
1248	2 374	26	2 400	4 053	0	0	0	4 053	6 427	6 453
1280	4 336	1	4 337	69 722	11	10	21	69 743	74 058	74 080
1536	32 279	15	32 294	7 715	93	78	171	7 886	39 994	40 180
2024	3 866	148	4 014	518	18	1 139	1 157	1 675	4 384	5 689
2048	1 145 150 629	957 926 391	2 103 077 020	6 237 496 955	785 124 454	811 142 851	1 596 267 305	7 833 764 260	7 382 647 584	9 936 841 280
2056	327	3	330	1 371	209	0	209	1 580	1 698	1 910
2064	213	1	214	4 636	836	98	934	5 570	4 849	5 784
2096	55	0	55	46 870	70	1	71	46 941	46 925	46 996
2432	11 863	417	12 280	8 144	462	156	618	8 762	20 007	21 042
3072	5 990 499	870 912	6 861 411	63 766 642	2 205 268	804 880	3 010 148	66 776 790	69 757 141	73 638 201
3248	76	0	76	2 057	173	1	174	2 231	2 133	2 307
4048	103	0	103	3 407	99	4	103	3 510	3 510	3 613
4072	103	2	105	2 140	35	0	35	2 175	2 243	2 280
4088	2 471	1	2 472	1 246	66	3	69	1 315	3 717	3 787
4096	165 217 560	1 731 928	166 949 488	1 103 360 301	81 417 342	48 805 544	130 222 886	1 233 583 187	1 268 577 861	1 400 532 675
8192	14 892	1 252	16 144	59 142	18 621	2 157	20 778	79 920	74 034	96 064
16384	862	49	911	568	58	83	141	709	1 430	1 620
All	1 469 801 301	983 785 024	2 453 586 325	7 450 427 415	874 021 161	863 826 493	1 737 847 654	9 188 275 069	8 920 228 716	11 641 861 394

Tab. 4.2.: Deduplication Summary. Contains statistics about how many certificates were removed and based on which duplicate collision.

As mentioned in [Section 3.3](#) we first processed the valid certificates, classifying them into valid *duplicate* and valid *unique*. For the invalid certificates, we had to do two lookups, starting with the *valid hash set*. If a collision was detected, then this certificate would be classified as *Duplicates Other*. Afterward we checked for collisions in the *invalid hash set*, classified as *Duplicates (self)* and finally if no collisions were detected the certificate was classified as *invalid unique*, see [Table 4.2](#).

Key Size	Intra			Inter			Summary	
	Valid	Invalid	Total	Valid	Invalid	Total	Overlap	Total
512	82	96 609	96 691	179	0	179	78	96 792
1024 ¹	3 615	29 069	32 684	1 950	0	1 950	633	34 001
1032	0	153	153	0	0	0	0	153
2048 ¹	6 294	281	6 575	13	0	13	8	6 580
2056	28	0	28	54	0	54	28	54
4096 ¹	2	8	10	1	0	1	0	11
All	10 021	125 120	136 141	2 197	0	2 197	747	137 591

¹TODO

Tab. 4.3.: Distribution GCD successes based on key size. Key sizes not depicted here had no successes.

4.3 GCD

As mentioned in [Section 2.5.1](#) we split the GCD computation into two parts, Intra-GCD and Inter-GCD. Intra-GCD computed divisors only between certificates of the same file, and Inter-GCD computed divisors between different files. Intra-GCD having a linear time complexity was fully executed. In comparison, Inter-GCD has a quadratic runtime, based on which we reduced the calculation scope and computed each key size separately: GCD was executed between a certificate from file x and the product of file y with both files x and y , containing only certificates of the same key size. Additionally, we focused only on valid certificates, which further reduced the required computation time.

With these concessions, we still could not compute all key sizes. Keys of size 1024 bit, 2048 bit, 3072 bit and 4096 bit only had 4%, 0.005%, 36% and 0.06% completed within the allotted time of one week, respectively. The reason for this is not only the quadratic runtime based on the amount of files, but also the exponential growth based on the key size, which resulted in an increased runtime for the GCD computation. This meant that it took around one second for a single GCD calculation for 1024-bit keys and around ten seconds for 4096-bit keys.

Our results included an overlap with the small-prime attack, see [Table 4.5](#), this leaves an option for a non-standard implementation of RSA where n consists of more than two primes. Given that none of these key belonged to valid certificates, we did not further analyze them and attributed them to test certificates. This would be an area for future work to discover where these keys came from and their impact.

¹This key size was not processed completely.

Based on the results in [Table 4.3](#) we approximate the undiscovered valid keys to total 310,450 as depicted in [Table 4.4](#). This is based on our limited calculation scope. The real amount of vulnerable certificates should be much larger when allowing for the mixing of key sizes and processing invalid certificates.

In the current results, most of the compromised certificates were found using Intra-GCD. Intra-GCD however is not sufficient, with Inter-GCD finding comparatively more results, for example the key size 1024 where Inter-GCD returned around half of the results of Intra-GCD with only four percent of the files being processed. Both parts of GCD contain an overlap where some certificates were marked by both methods, there were however always certificates which were detected by only one of them, making the methodologies non-exclusive. In total 386,641 compromised certificates were discovered.

Key Size	Inter valid
1024	$\approx 48\,750$
2048	$\approx 260\,000$
3072	≤ 5
4096	$\approx 1\,700$
Total	$\approx 310\,450$

Tab. 4.4.: Estimation for inter calculation of valid certificates.

4.4 Attacks

Due to hardware and time limitations, not all attacks could be applied. Therefore, we chose all attacks that would take less than **1s** per certificate as seen in [Figure 4.1](#), these are located in the upper diagram. This allowed us to execute these attacks on **all** RSA certificates. Additionally, we included a block-list approach for leaked or already compromised certificates. We use the blocklists provided by [[@Böc25](#)]. This includes leaked keys from webkeys, gitkeys, fortikeys, pkgkeys as well as known debianopenssl bug keys. The lookup time for a certificate would be correlated to the time it takes to hash the public key.

In [Figure 4.1](#) the *throughput* attack is a baseline consisting only of processing overhead. There are three different implementations of the ROCA test, all of which perform fast enough to be used and all of which lead to the same results. Some attacks have a GCD variant, these work by generating the corresponding prime

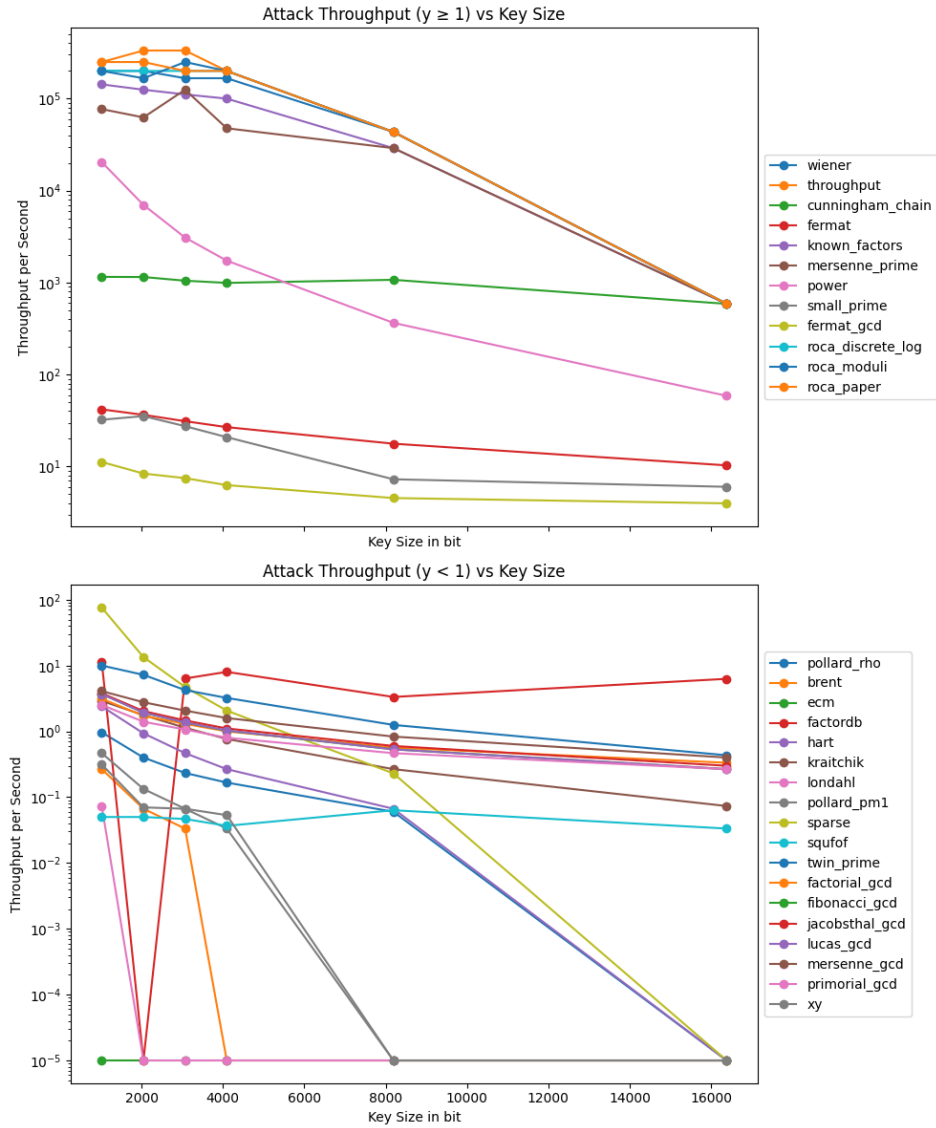


Fig. 4.1.: Attack Benchmark. Split all tested attacks into those that take more and less than one second per certificate. Each attack had a timeout of five minutes to complete at least one attack.

and then checking if it is a common divisor of the attacked key. Lastly, due to the logarithmic scale if an attack had a throughput of zero after the timeout of five minutes, then the throughput was set at 10^{-5} for visualization purposes (Logarithmic Scale).

As seen in [Table 4.5](#) the most successful attack came from the Debian- OpenSSL blocklist, however most of these certificates were already expired and the remaining one did not have a valid chain of trust. This is presumably caused by the fame of the underlying openssl bug. Fermat was the second best attack and contained

Attack Name	Valid	Invalid	Total	Unique	Duplicate
debianopenssl.csv blocklist	1 923	14 228	16 151	16 151	0
fermat 100	13 841	27	13 868	13 868	0
small prime product 1mil	2 871	1 361	4 232	1 779	2 453
ROCA	443	2 769	3 212	3 212	0
mersenne prime	1 606	755	2 361	0	2 361
gitkeys.csv blocklist	407	578	985	985	0
fermat gcd product	189	201	390	0	390
debianopensslsmall.csv blocklist	4	296	300	300	0
webkeys.csv blocklist	6	16	22	22	0
cunningham chain	0	2	2	0	2
known factors	0	1	1	1	0
keypairvuln.csv blocklist	0	1	1	1	0
All	21 290	20 235	41 525	36 319	5 206

Tab. 4.5.: Distribution of successful attacks based on attack name. Key sizes not depicted, had no successes.

mostly valid certificates. The "small prime product 1mil" attack attempted to find a common divisor between each certificate and the product of the first million primes, having moderate success. ROCA similar to debian openssl also had its majority in the invalid certificates.

Additionally, [Table 4.5](#) contains a further breakdown of the results, looking at the uniqueness of each attack. The only overlap that appeared is between "small prime", "mersenne prime", "fermat gcd" and "cunningham chain". This was traced back to small primes being detected by all these attacks, leading to "small prime" containing results from all other attacks, effectively superseding the others.

4.5 Statistics

After collecting the results of GCD and singular attacks, we reconstructed the original certificates from the (n, e) pairs. The total amount of certificates compromised by both GCD and singular attacks totals to 174,745 as seen in [Table 4.6](#). The majority of the compromised certificates were detected by GCD, with most being invalid at the time of discovery. Given the validity timeframe of a certificate to be at maximum

Key Size	Valid	Invalid	Total
512	190	96 922	97 112
528	0	1	1
768	0	1	1
1024	7 046	41 456	48 502
1032	1	154	155
1920	0	96	96
2048	22 102	5 912	28 014
2056	61	3	64
2208	0	1	1
2432	1	1	2
3072	17	19	36
4088	1	0	1
4096	177	582	759
8192	3	3	6
All	29 594	145 151	174 745

Tab. 4.6.: Compromised certificate summary

a year, all of these certificates would be invalid by the time of publication. In total **8346** certificates were at some point in time valid.

4.6 Proof of Concept faked Certificate

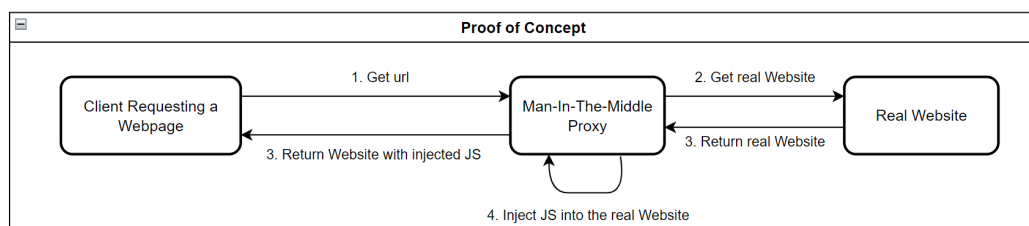


Fig. 4.2.: Proof of concept diagram.

To highlight the difference between a real web server and a compromised one, we developed a small demonstration. It consists of two parts, a python flask web server, which acts as the malicious server, and a DNS spoofed web browser. Spoofing the web browser is done using Chromium command-line option which allows us to

redirect traffic from a specified domain name to a different IP: **chromium --host-resolver-rules="MAP tu.berlin 127.0.0.1"** We use this to redirect traffic from the real web server to localhost, which runs our malicious web server. The malicious server works as proxy by requesting the original homepage and injecting *malicious* JavaScript into it. Since our server has the same web certificate as the original server, the browser cannot detect our intervention and considers the connection secure. The connection flow can be seen in [Figure 4.2](#).

By the time we finished processing the downloaded certificates, none were currently active. Looking at the most recently active certificates, we discovered *backloop.dev* for which we reran GCD and recovered the private key. However, given that this website openly displays its private key, we feel that sharing it as part of our proof of concept is acceptable. You can see our injected JavaScript code in [Figure 4.3](#) with Chromium believing that our proxy is the real website.

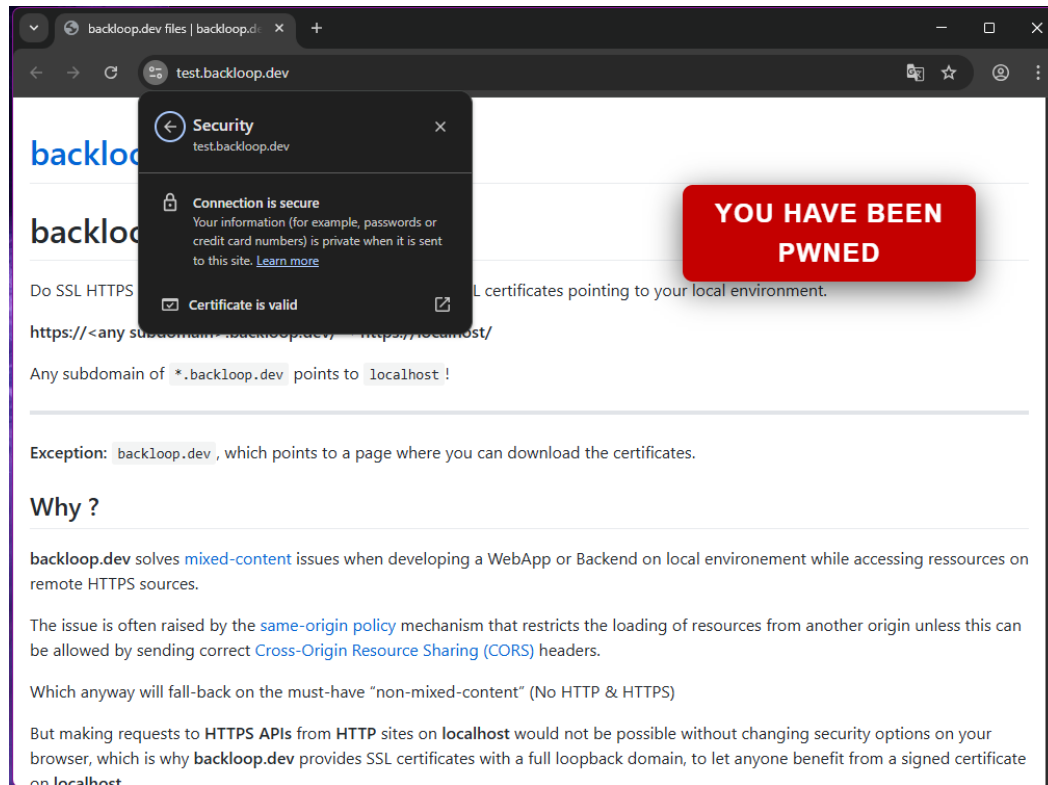


Fig. 4.3.: Proof of Concept using backloop.dev

Discussion

The results of this work show a decreased effectiveness of known cryptographic attacks on RSA keys. Across ≈ 11.6 billion certificates, singular attacks such as Fermat factorization and ROCA testing yielded comparatively few compromised keys. In contrast, greatest common divisor (GCD) was responsible for the majority of discovered vulnerabilities, both in absolute numbers and in their ability to compromise previously secure certificates.

5.1 Relevance of Singular Attacks

Singular attacks identified 36,319 compromised certificates, most of which fully factorized the modulus, except for small prime, that recovered partial factorizations containing any of the first million of primes. While not insignificant, this confirms that most historical weaknesses in RSA key generation have been addressed. The best-performing singular attack in terms of valid certificates was Fermat factorization, suggesting that some poorly generated keys continue to appear.

Blocklist-based approaches looked up certificates in a set of compromised keys containing those generated by the OpenSSL bug or keys leaked in hacks. Consequently, blocklists function as a sanity-check for in-use certificates, verifying that leaked keys were deactivated and known issues fixed.

If singular attacks were restricted to currently valid certificates, many could be safely omitted with minimal loss of coverage. Blocklists would remain useful, and Fermat could still uncover edge cases. Overall, singular attacks appear to have diminishing returns in the modern certificate ecosystem.

5.2 Effectiveness and Limitations of GCD-Based Analysis

GCD identified 386,641 compromised certificates. This suggests that shared-prime vulnerabilities remain the dominant weakness in RSA. This effectiveness, however, comes with significant computational requirements. Even under a partial analysis limited to valid certificates, separate key sizes, and incomplete Inter-file execution, the runtime requirements were unfeasible for a complete analysis. For 2048-bit keys, a complete Inter-GCD computation would require decades on thousands of CPU cores.

These findings confirm that while GCD can discover a lot of vulnerable RSA keys, it is not suitable for ecosystem-wide scanning.

5.3 Targeted GCD as a Practical Threat Model

Despite inefficiency at a large scale, GCD is usable in targeted attack scenarios. Once a comprehensive dataset of deduplicated public keys has been downloaded, requiring ≈ 2.7 TB of storage for the compressed version, an adversary can test individual target keys against the entire dataset at a comparatively low cost.

To approximate the time required, let us assume a GCD calculation takes 10 seconds on a single core (based on benchmarks of 4096-bit keys) and computing a product from 1 million keys takes around 140 seconds (based on tree-multiplication benchmark). To process a key we would need to deduplicate it first, this would take around 10 hours based on a read speed of 150MB/s. Afterward, we would run GCD with that key and the whole dataset. Considering 9 thousand files each of size 1 million, this would result in 375 hours or around 15 days to process a single key (16 with deduplication). Assuming a system with 8-cores, this drops to ≈ 2 days.

Importantly, computation time scales **partially** with the number of target keys, each key adding approximately a day of computation time on a 8 core CPU. This makes small batches of targets economically attractive. Certificate Authority, or intermediate certificates, represent particularly prized targets, since a single compromise would undermine trust for large portions of the internet.

This positions GCD as a selective, high-impact attack. One that is usable not only by large-scale organizations, but also any motivated individuals with commodity hardware.

5.4 Optimization

There are two aspects which can reduce the required processing time. First is the downloader, specifically because it was written in Python, it was throttled more by its MPSC channels (Writer Queue) than expected. Switching to a different language would reduce these issues and should increase processing speed.

Second of all hardware, although we scaled computation to over 2 thousand cores with their respective ALUs, complete results would still take over a decade. A different option would be to use FPGAs or ASICs which can be tailored for the large divisions part of GCD, increasing throughput by orders of magnitude. This however requires highly specialized hardware, whose design, development, and procurement are non-trivial and might be less cost-effective than just using more CPUs.

5.5 Web PKI Implications

Of the discovered certificates, 8,346 were once in use. This highlights that exploitation is not only possible, but also realistic. Additionally, the proof-of-concept demonstrates that once a private key is recovered, simple man-in-the-middle attacks can be executed without triggering any security warnings. The continued certificate lifetime reduction [[@Dav25](#)] decreases the absolute risk per certificate. However, it also simultaneously amplifies the risk for key generation infrastructure and monetarily important targets. Scaling automated issuance amplifies the consequences of systemic entropy failures, making rare but correlated key-generation flaws more likely.

This opens the possibility of CAs testing the security of RSA keys before a certificate is issued to them. As mentioned previously, we cannot expect GCD to be tested on every RSA key due to the computational constraints. Singular attacks however are much more realistic since they can be easily scaled and require less computation. If selecting ROCA, Fermat, and small primes this would result in a estimated 100 ms which would be similar to the time it takes to generate an RSA key as seen in [[@Rsc24](#)].

5.6 Datasets

Although this work used the Censys dataset consisting of Web Certificates with RSA public keys, the methodology described is applicable to any RSA key. Future work could therefore expand on the data collected to include other sources of RSA keys such as those used in SSH. A possible source could be GitHub [git25], which allows users to host their public keys for easy retrieval and simple integration when configuring trusted keys on servers. Using their API, it is possible to retrieve the public keys of all their users. Nonetheless, given the prevalent usage of tools such as `ssh-keygen` which by default do not generate RSA keys, further research is required toward the relevance of such datasets.

5.7 ECDSA

A significant part of the dataset consisted of ECDSA keys. Analyzing this key type would mostly require additional information besides just the public key. Therefore, we would need to develop a new system which can acquire the additional information and then process the keys which would be beyond the scope of this work.

5.8 Ethics

While the results demonstrate that GCD-based attacks are currently the most promising method for compromising RSA keys at scale, they also show that such attacks require extensive computational resources, particularly when applied indiscriminately.

Crucially, ongoing changes limit the real-world impact. The reduction of certificate validity periods narrows the exploitation window. Increasing key sizes increase computation time, and switching from RSA to other key types such as ECDSA completely removes these attack-vectors. Even if a certificate is compromisable, the time required to discover it may exceed the certificate's lifetime, rendering any success meaningless.

There are however ethical questions about the public disclosure of targeted GCD attacks, particularly when it comes to high-value infrastructure keys. Therefore, care must be taken when disclosing methodological insights and aggregate statistics to

prevent unintentionally facilitating abuse. Because of this, no private key material was reconstructed beyond controlled proof-of-concept demonstrations.

5.9 Outlook

This work demonstrates that targeted attacks are possible, but require extensive computational resources to execute at scale. Given the limited attack surface, high-impact certificates should be strictly analyzed before being deployed.

Compared to RSA, elliptic-curve or post-quantum alternatives would provide stronger systemic guarantees. Until then, GCD continues to be a high-impact attack vector that exploits rare systemic failures in key generation rather than cryptographic weakness itself.

Conclusion

In this work, we conducted a large-scale analysis of RSA certificates targeted at identifying compromised keys. We used known cryptographic attacks and an adaptation of GCD to discover 386,641 compromised certificates. These results were achieved by processing the Censys, which contained more than 11.6 billion certificates. We first deduplicated the dataset based on public-key values, which reduced the certificated to around 8.9 billion keys. Afterward we applied GCD, split into Intra- and inter-file variants, and known attacks. While Intra-GCD could be executed fully, Inter-GCD was limited due to its quadratic time complexity. As a result, only a fraction of larger key sizes could be processed. Despite these limitations, GCD proved to be the most effective method, having contributed the majority of our results.

In parallel, we evaluated per-certificate attacks, including Fermat factorization, ROCA detection, small-prime-check, as well as a blocklist of leaked keys. These attacks were applied across the full dataset. The Blocklist-method and Fermat factorization identified a non-negligible number of compromised keys, all of which were however invalid at the time of discovery.

Finally, we demonstrated the practical impact of compromised web-certificates through a proof-of-concept demonstration, revealing that man-in-the-middle attacks remain undetectable to modern browsers given a valid private key. Overall, our results show that known singular attacks have limited relevance for the current PKI, while large-scale datasets combined with targeted GCD still pose a serious threat.

Bibliography

- [Ber04] Daniel J Bernstein. “How to find smooth parts of integers”. In: URL: <http://cr.yp.to/papers.html#smoothparts>. ID 201a045d5bb24f43f0bd0d97fcf5355a. Citations in this document 20 (2004) (cit. on pp. 9, 16).
- [Cli] ClickHouse. *Fast open-source OLAP DBMS* (cit. on p. 17).
- [Cop96] Don Coppersmith. “Finding a Small Root of a Univariate Modular Equation”. In: *Advances in Cryptology — EUROCRYPT ’96*. Ed. by Ueli Maurer. Berlin, Heidelberg: Springer Berlin Heidelberg, 1996, pp. 155–165 (cit. on p. 8).
- [git25] github. *GitHub*. 2025 (cit. on p. 30).
- [Hen+12] Nadia Heninger, Zakir Durumeric, Eric Wustrow, and J Alex Halderman. “Mining your Ps and Qs: Detection of widespread weak keys in network devices”. In: *21st USENIX Security Symposium (USENIX Security 12)*. 2012, pp. 205–220 (cit. on p. 1).
- [Len+12] Arjen K Lenstra, James P Hughes, Maxime Augier, et al. “Ron was wrong, Whit is right”. In: *Cryptology ePrint Archive* (2012) (cit. on p. 1).
- [MVV18] Alfred J Menezes, Paul C Van Oorschot, and Scott A Vanstone. *Handbook of applied cryptography*. CRC press, 2018 (cit. on p. 9).
- [Nat23] National Institute of Standards and Technology (US). *Digital Signature Standard (DSS)*. Tech. rep. Washington, D.C., Feb. 2023 (cit. on pp. 7, 8).
- [Nem+17] Matus Nemec, Marek Sys, Petr Svenda, Dusan Klinec, and Vashek Matyas. “The Return of Coppersmith’s Attack: Practical Factorization of Widely Used RSA Moduli”. In: *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. CCS ’17. Dallas, Texas, USA: Association for Computing Machinery, 2017, pp. 1631–1648 (cit. on pp. 1, 9).
- [Res18a] Eric Rescorla. *The Transport Layer Security (TLS) Protocol Version 1.3*. RFC 8446. Aug. 2018 (cit. on p. 5).
- [Wie90] Michael J. Wiener. “Cryptanalysis of Short RSA Secret Exponents”. In: *Advances in Cryptology — EUROCRYPT ’89*. Ed. by Jean-Jacques Quisquater and Joos Vandewalle. Berlin, Heidelberg: Springer Berlin Heidelberg, 1990, pp. 372–372 (cit. on p. 8).

Webpages

- [@AD99] Christopher Allen and Tim Dierks. *The TLS Protocol Version 1.0*. Jan. 1999. URL: <https://www.rfc-editor.org/info/rfc2246> (cit. on p. 6).
- [@Apa25] Apache. *Apache Avro serialization format for record data*. Dec. 2025. URL: <https://avro.apache.org/> (cit. on p. 14).
- [@Bar+15] Richard Barnes, Martin Thomson, Alfredo Pironti, and Adam Langley. *Deprecating Secure Sockets Layer Version 3.0*. June 2015. URL: <https://www.rfc-editor.org/info/rfc7568> (cit. on p. 6).
- [@Bel08] Luciano Bello. *The Debian Project. openssl – predictable random number generator*. May 2008. URL: <http://www.debian.org/security/2008/dsa-1571> (cit. on p. 1).
- [@Böc25] Hanno Böck. *Badkeys*. Dec. 2025. URL: <https://badkeys.info/> (cit. on p. 22).
- [@Boe+08] Sharon Boeyen, Stefan Santesson, Tim Polk, et al. *Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile*. May 2008. URL: <https://www.rfc-editor.org/info/rfc5280> (cit. on pp. 3, 4).
- [@Cen25] Censys. Dec. 2025. URL: <https://search.censys.io/> (cit. on pp. 11, 13).
- [@Dav25] Stephen Davidson. *TLS certificate lifetimes will officially reduce to 47 days*. May 2025. URL: <https://www.digicert.com/blog/tls-certificate-lifetimes-will-officially-reduce-to-47-days> (cit. on pp. 1, 29).
- [@DR06] Tim Dierks and Eric Rescorla. *The Transport Layer Security (TLS) Protocol Version 1.1*. Apr. 2006. URL: <https://www.rfc-editor.org/info/rfc4346> (cit. on p. 6).
- [@Hes+05] Peter Hesse, Matt Cooper, Yuriy A. Dzambasow, Susan Joseph, and Richard Nicholas. *Internet X.509 Public Key Infrastructure: Certification Path Building*. Sept. 2005. URL: <https://www.rfc-editor.org/info/rfc4158> (cit. on pp. 3, 4).
- [@Mor+16] Kathleen Moriarty, Burt Kaliski, Jakob Jonsson, and Andreas Rusch. *PKCS #1: RSA Cryptography Specifications Version 2.2*. Nov. 2016. URL: <https://www.rfc-editor.org/info/rfc8017> (cit. on p. 7).
- [@PT11] Tim Polk and Sean Turner. *Prohibiting Secure Sockets Layer (SSL) Version 2.0*. Mar. 2011. URL: <https://www.rfc-editor.org/info/rfc6176> (cit. on p. 6).
- [@PPR01] Nick Pope, Denis Pinkas, and John Ross. *Electronic Signature Policies*. Sept. 2001. URL: <https://www.rfc-editor.org/info/rfc3125> (cit. on pp. 3, 4).
- [@Res18b] Eric Rescorla. *The Transport Layer Security (TLS) Protocol Version 1.3*. Aug. 2018. URL: <https://www.rfc-editor.org/info/rfc8446> (cit. on p. 6).

- [@RD08] Eric Rescorla and Tim Dierks. *The Transport Layer Security (TLS) Protocol Version 1.2*. Aug. 2008. URL: <https://www.rfc-editor.org/info/rfc5246> (cit. on p. 6).
- [@Rsc24] Rsc. *Crypto RSA: New key generation prohibitively slow under race detector* · ISSUE 70644 · Golang Go. Dec. 2024. URL: <https://github.com/golang/go/issues/70644#issuecomment-2524072173> (cit. on p. 29).
- [@San+13] Stefan Santesson, Michael Myers, Rich Ankney, et al. *X.509 Internet Public Key Infrastructure Online Certificate Status Protocol - OCSP*. June 2013. URL: <https://www.rfc-editor.org/info/rfc6960> (cit. on p. 4).
- [@Shi07] Robert W. Shirey. *Internet Security Glossary, Version 2*. Aug. 2007. URL: <https://www.rfc-editor.org/info/rfc4949> (cit. on p. 6).
- [@Sho25] Shodan. Dec. 2025. URL: <https://www.shodan.io/search?query=tls> (cit. on p. 11).

List of Figures

3.1	Pipeline Overview. Supports DSA and ECDSA analysis with a focus on RSA.	13
3.2	A Generic Pipeline. Consists of a file queue being fed into reader threads. These reader threads read files and split them into separate certificates. Each certificate gets passed through a different queue to the worker threads. Worker threads analyze or process the certificate. If the processing has a positive result, then the certificate and result both get sent to a writer to save to disk.	14
3.3	Downloader Pipeline. Based on the generic pipeline with a different method of sending processed certificates from workers to writers. . . .	14
3.4	Deduplication pipeline. Based on the generic pipeline with an additional processing context in the workers.	15
4.1	Attack Benchmark. Split all tested attacks into those that take more and less than one second per certificate. Each attack had a timeout of five minutes to complete at least one attack.	23
4.2	Proof of concept diagram.	25
4.3	Proof of Concept using backloop.dev	26
A.1	Downloader Architecture	41
A.2	Downloader Worker.	41
A.3	Downloader Writer.	42
A.4	Downloader Workflow.	42
A.5	Deduplicator Architecture.	42
A.6	Deduplicator Worker.	42
A.7	Deduplicator Writer.	43
A.8	Deduplicator Workflow.	43
A.9	Singular Attack Architecture.	43
A.10	Singular Attack Reader.	43
A.11	Singular Attack Worker.	44
A.12	Singular Attack Writer.	44
A.13	Singular Attack Workflow.	44

A.14	Intra-GGT Architecture.	45
A.15	Inter-GGT Workflow.	45
A.16	Intra-GGT Worker.	45

List of Tables

4.1	Certificate Download Summary. It shows the three types of downloaded certificates, split into valid and invalid based on the download date of 30.05.2025 and the certificates <i>Validity</i> time-frame.	19
4.2	Deduplication Summary. Contains statistics about how many certificates were removed and based on which duplicate collision.	20
4.3	Distribution GCD successes based on key size. Key sizes not depicted here had no successes.	21
4.4	Estimation for inter calculation of valid certificates.	22
4.5	Distribution of successful attacks based on attack name. Key sizes not depicted, had no successes.	24
4.6	Compromised certificate summary	25

Appendix

A.1 System architecture

In this section, we visualize additional details about our systems. This includes system design as well as flow diagrams.

A.1.1 Downloader

This section details the inner workings of the Downloader with additional details relating to the worker, writer, and generic execution flow.

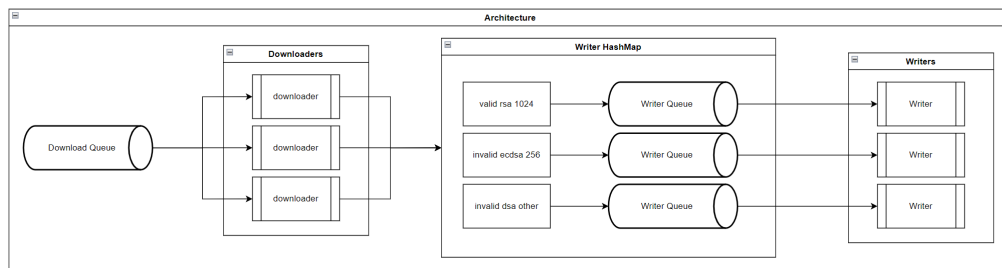


Fig. A.1.: Downloader Architecture

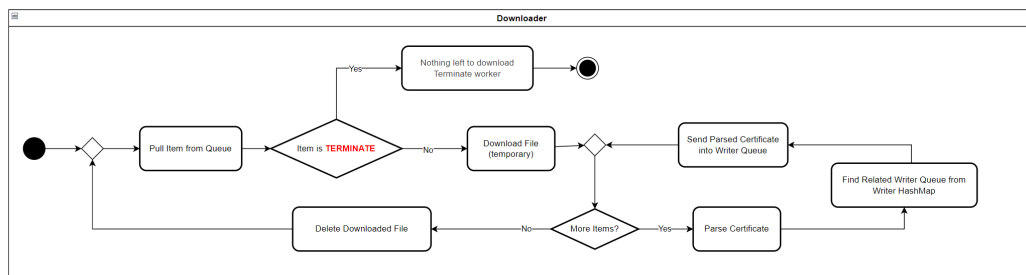


Fig. A.2.: Downloader Worker.

A.1.2 Deduplicator

This section details the inner workings of the Deduplicator with additional details relating to the worker, writer, and generic execution flow.

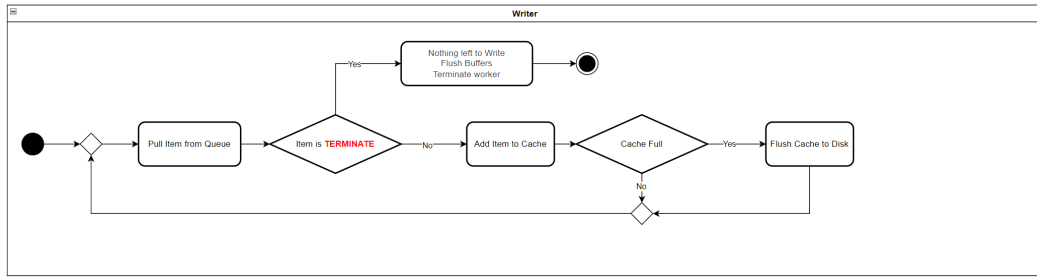


Fig. A.3.: Downloader Writer.

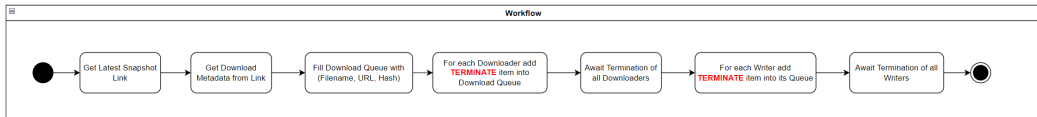


Fig. A.4.: Downloader Workflow.

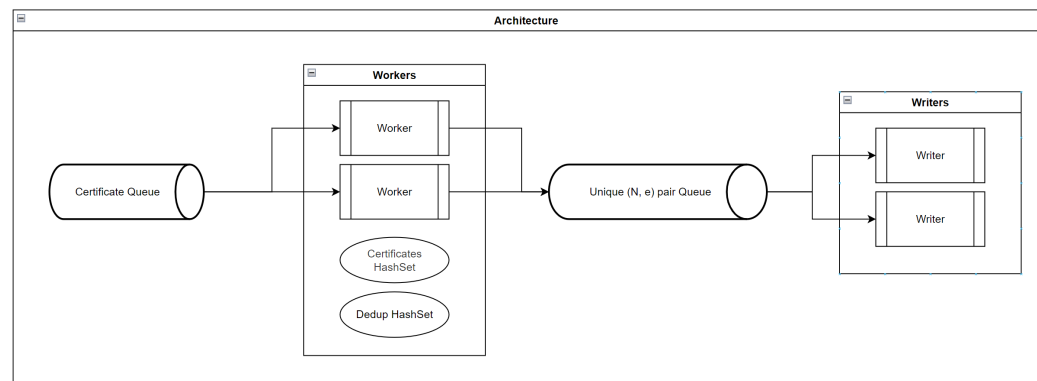


Fig. A.5.: Deduplicator Architecture.

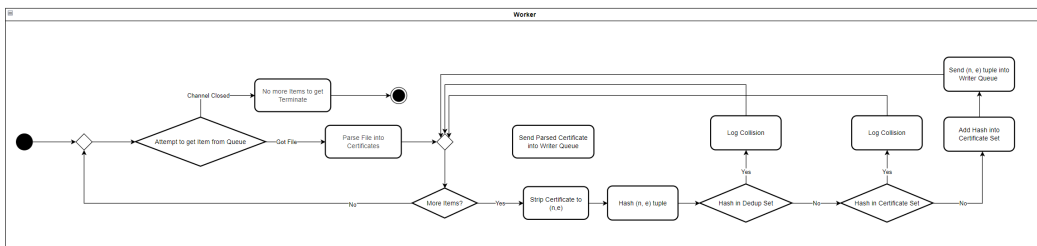


Fig. A.6.: Deduplicator Worker.

A.1.3 Singular Attack Pipeline

This section details the inner workings of the Singular Attack Pipeline with additional details relating to the reader, worker, writer, and generic execution flow.

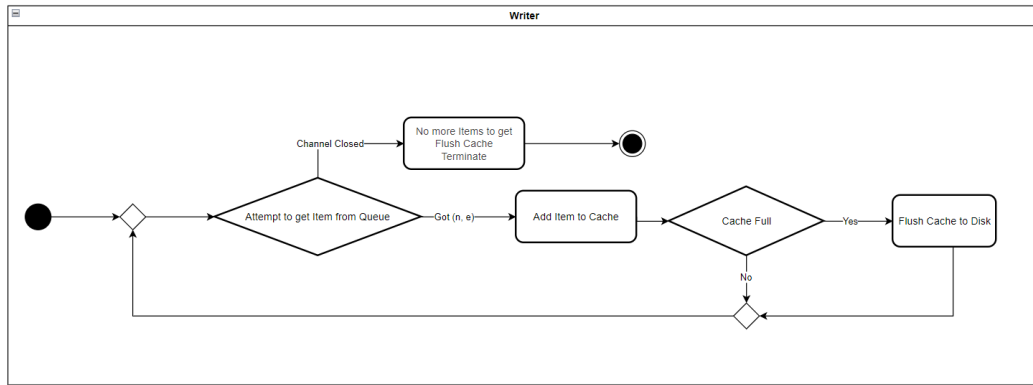


Fig. A.7.: Deduplicator Writer.

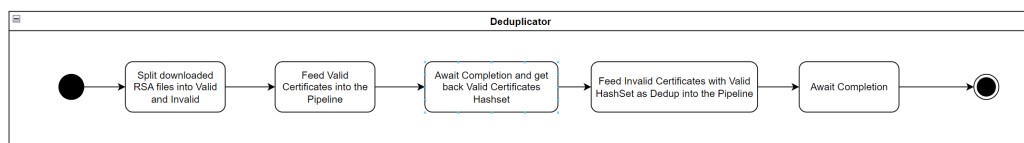


Fig. A.8.: Deduplicator Workflow.

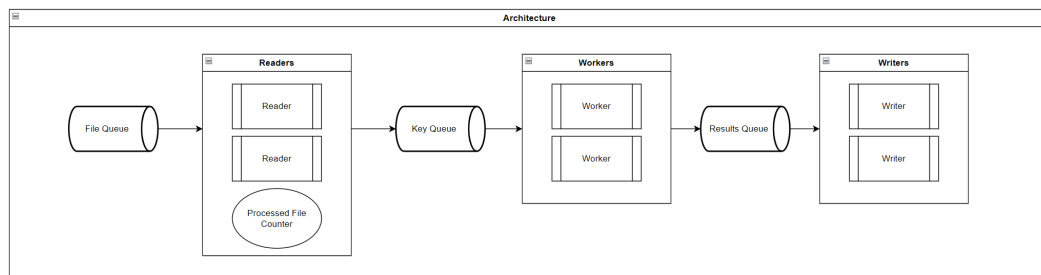


Fig. A.9.: Singular Attack Architecture.

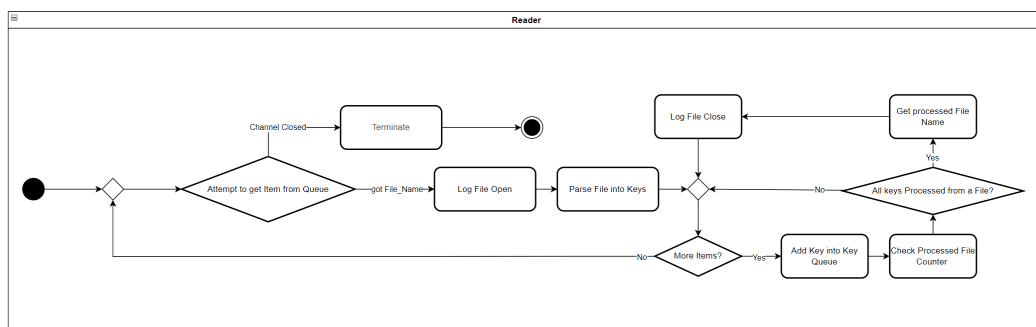


Fig. A.10.: Singular Attack Reader.

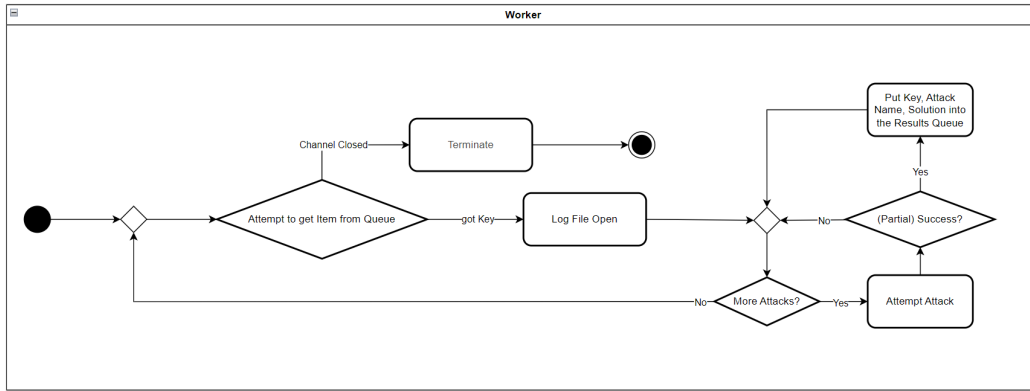


Fig. A.11.: Singular Attack Worker.

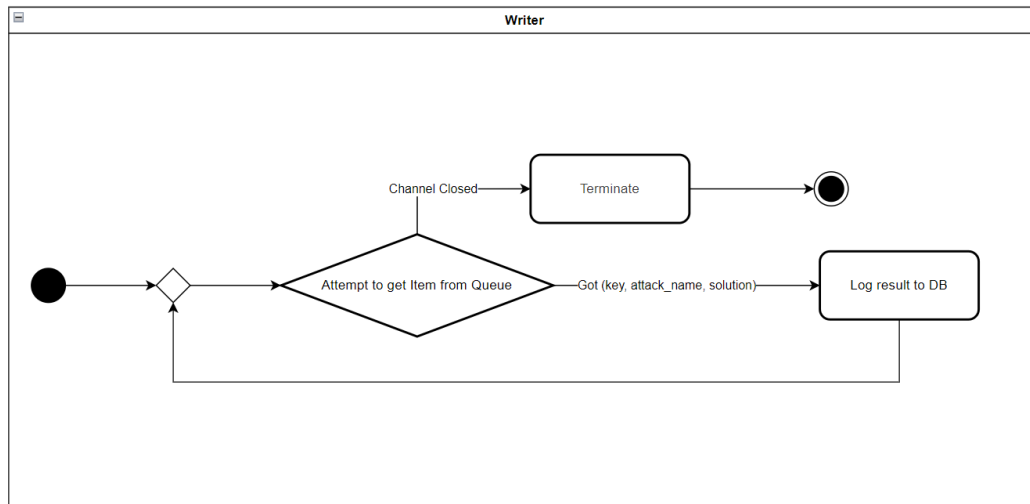


Fig. A.12.: Singular Attack Writer.

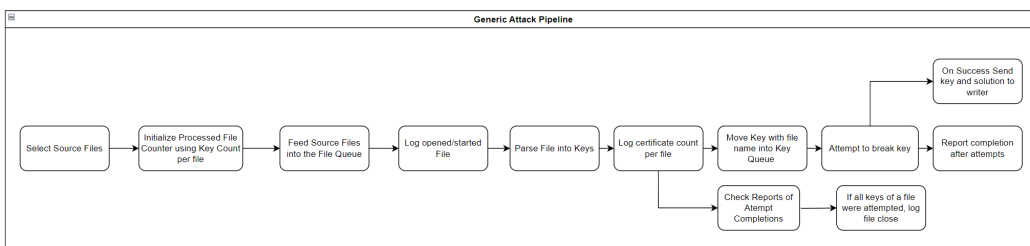


Fig. A.13.: Singular Attack Workflow.

A.1.4 GGT Pipelines

This section details the inner workings of the GGT Pipelines with additional details relating to the architecture workflow and worker.

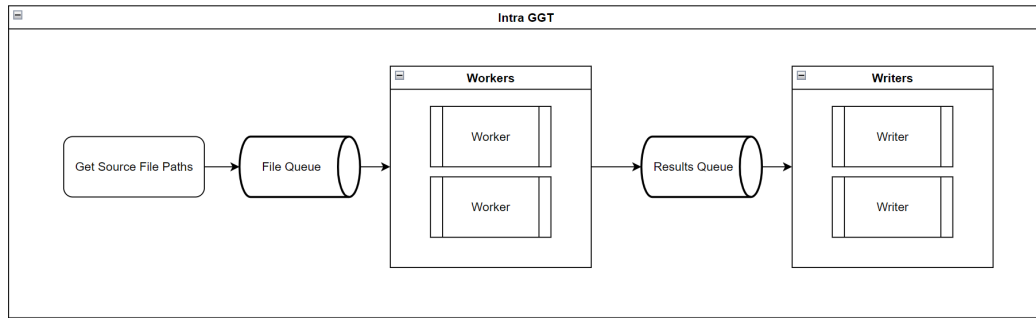


Fig. A.14.: Intra-GGT Architecture.

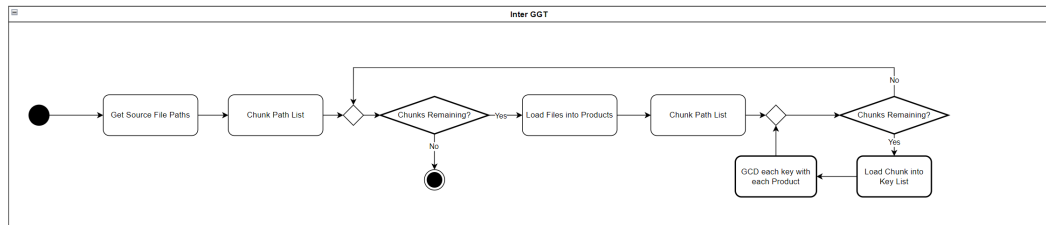


Fig. A.15.: Inter-GGT Workflow.

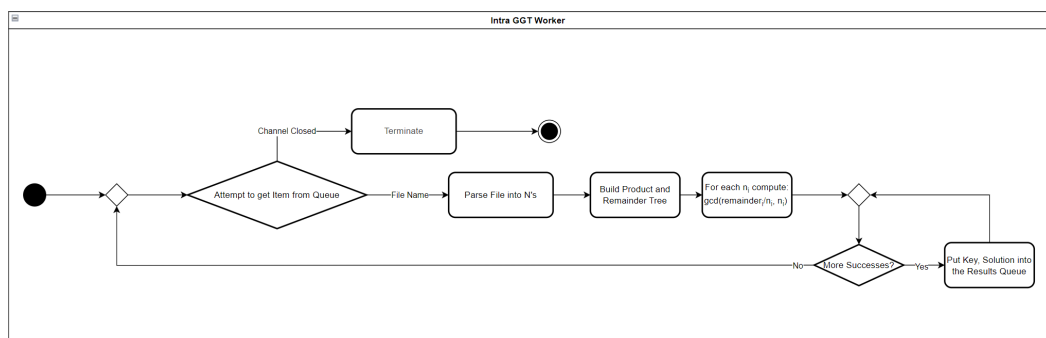


Fig. A.16.: Intra-GGT Worker.